CSE 202

Algorithm Design and Analysis

Russell Impagliazzo

Thanks, Qipeng Liu

Miles Jones

Today’s plan

- Greedy algorithms introduction
- Event scheduling
- Exchange argument

The Greedy Method

- Many problems involve finding a globally best solution.
- The entire solution can often be thought of as a combination of smaller ``local decisions''. (This idea also used in back-tracking, dynamic programming, hill-climbing, etc.)
- A greedy algorithm repeatedly makes what seems to be a ``best'' local decision, until all decisions are made.
- This is called the Greedy Method.
- It doesn't always work.
- Just as in life, acting in one's immediate best interest is not always the best longer-term strategy.

The Greedy Method

- The Greedy Method does not always work. Because of this, in order to use the Greedy Method, we must prove the correctness of the algorithm.

- Or else, we must present a counterexample to show that a particular greedy method will not work.

The Greedy Method correctness

Correctness means finding an optimal solution.
When are greedy algorithms correct?

Intuition: Greedy algorithms need to balance the
direct benefit of a decision
with the opportunity costs

Why proving correctness is tricky

- Need to compare the solution our algorithm finds to a global optimum
- We know something about our algorithm's solution
- We don't know anything about the hypothetical global optimum except that it must meet the constraints.

How do we compare a known to an unknown?

Fortunately, standard templates exist

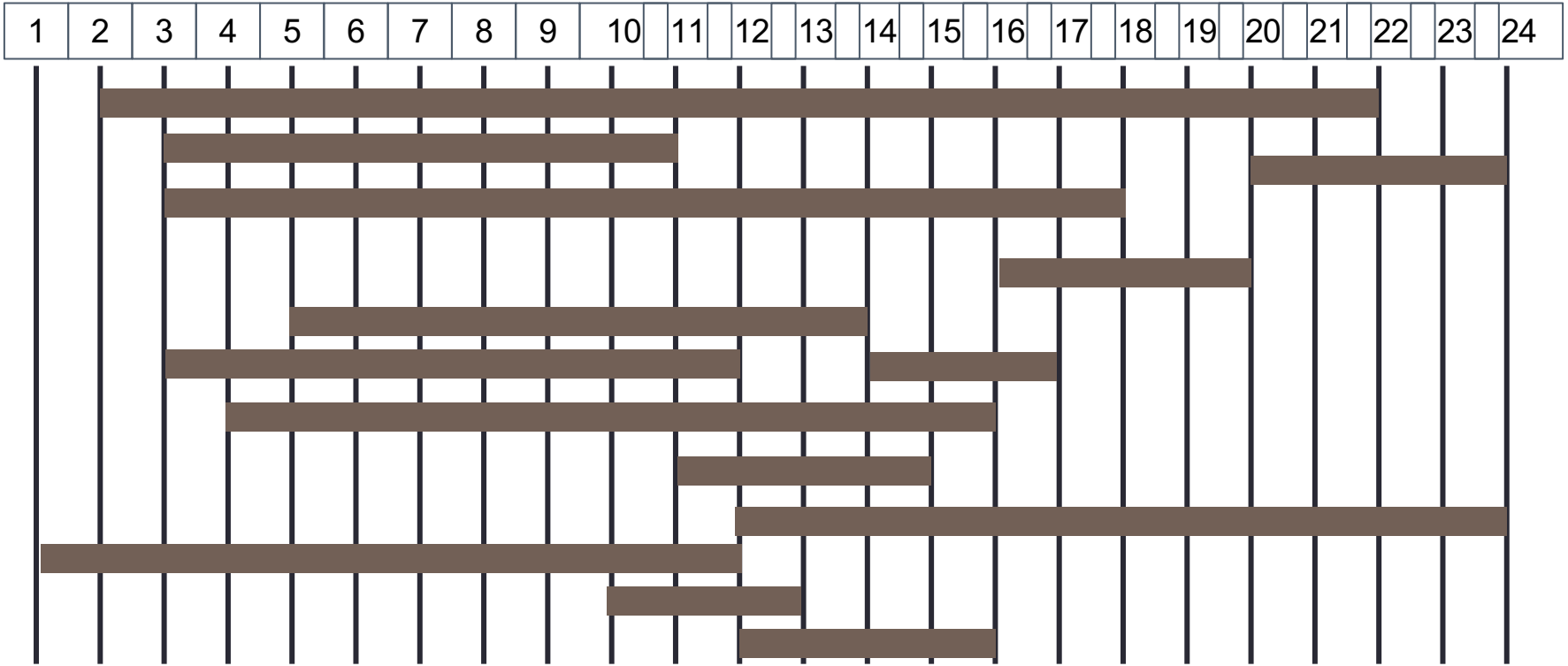
- Exchange or modification method**
- Greedy stays ahead: simplest when it works
- Achieves the bound: useful for approximation, local optimization
- Unique or canonical local optimum : nice when it works, also works for local optimization.

Event scheduling

- Suppose you are running a conference and you have a collection of events (or talks) that each have a start time and an end time.

  You only have one conference room!!!!
- Your goal is to schedule the most events possible that day such that no two events overlap. (We'll consider other variations later in the quarter.)

Event scheduling



Event scheduling SPECIFICATION

- Instance: Events[1...n], $Event_I =$
- Solution format:
- Constraints:
- Objective:

Event scheduling SPECIFICATION

- Instance: Events[1...n], $Event_I = (s_I, f_I), s_I < f_I$
- Solution format: List of events to be scheduled, in order of time:
- Constraints:
- Objective:

Event scheduling SPECIFICATION

- Instance: Events[1...n], $Event_I = (s_I, f_I), s_I < f_I$
- Solution format: List of events to be scheduled, in order of time:$i_1, i_2, \ldots . i_k$
- Constraints:
- Objective:

Event scheduling SPECIFICATION

- Instance: Events[1...n], $Event_I = (s_I, f_I), s_I < f_I$
- Solution format: List of indices of events $i_1, i_2, \ldots i_k$
- Constraints: $f_{i_1} \leq s_{i_2} < f_{i_2} \leq s_{i_3}, \ldots . f_{i_{k-1}} \leq s_{i_k}$
- Objective:

Event scheduling SPECIFICATION

- Instance: Events[1...n], $Event_I = (s_I, f_I), s_I < f_I$
- Solution format: List of indices of events $i_1, i_2, \ldots i_k$
- Constraints: $f_{i_1} \leq s_{i_2} < f_{i_2} \leq s_{i_3}, \ldots . f_{i_{k-1}} \leq s_{i_k}$
- Objective: maximize k, the number of scheduled events

Event scheduling

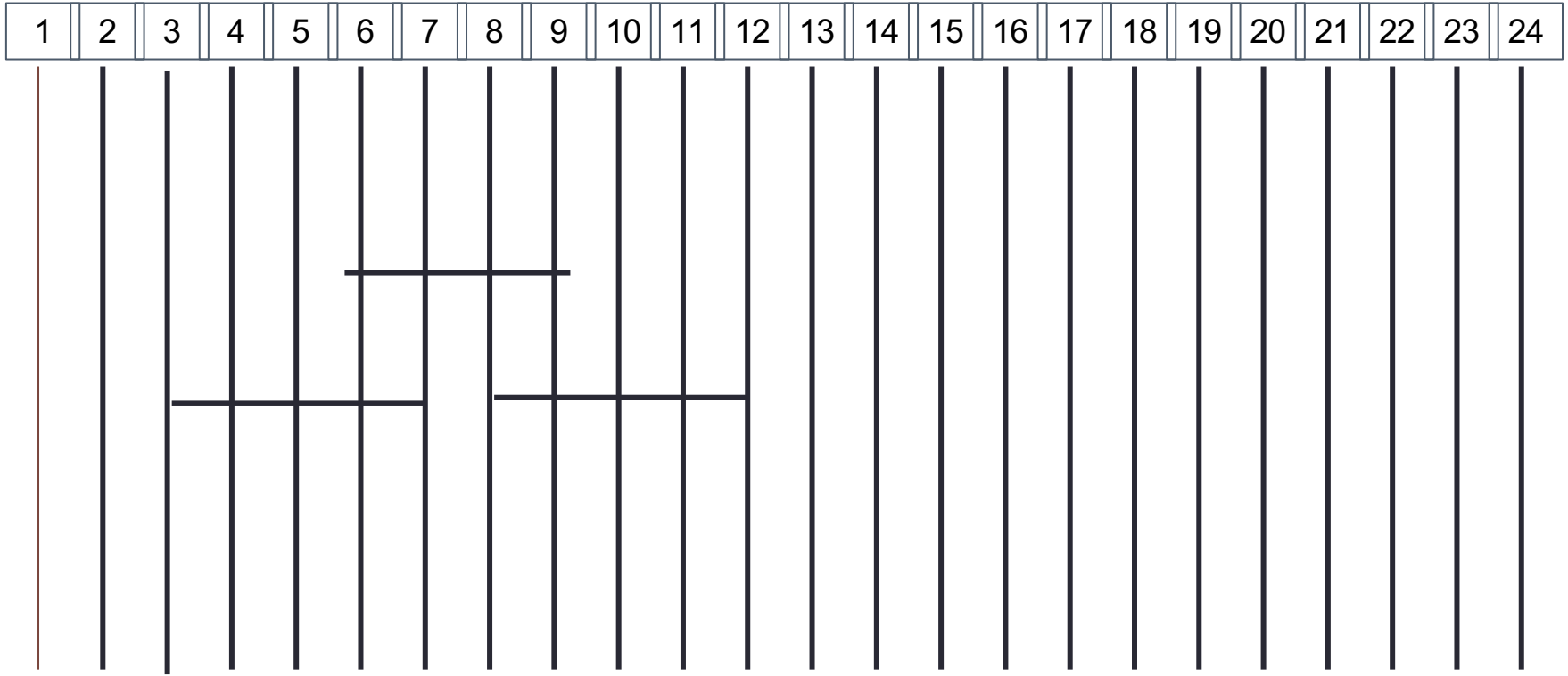
- Brute force is too slow. Let’s try some greedy strategies:

What rule should we use for deciding which event to pick first?

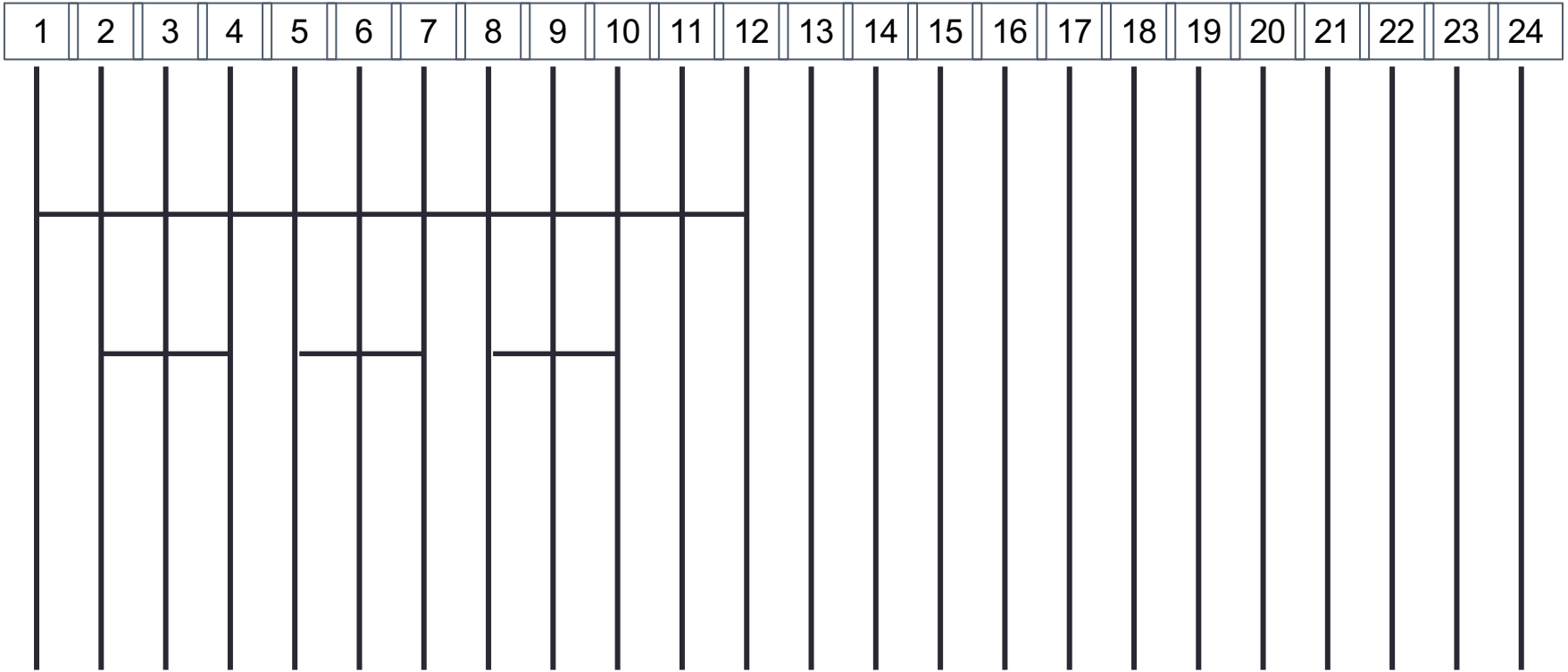
Event scheduling

- Brute force is too slow. Let’s try some greedy strategies:
  - Shortest duration
  - Earliest start time
  - Fewest conflicts
  - Earliest end time

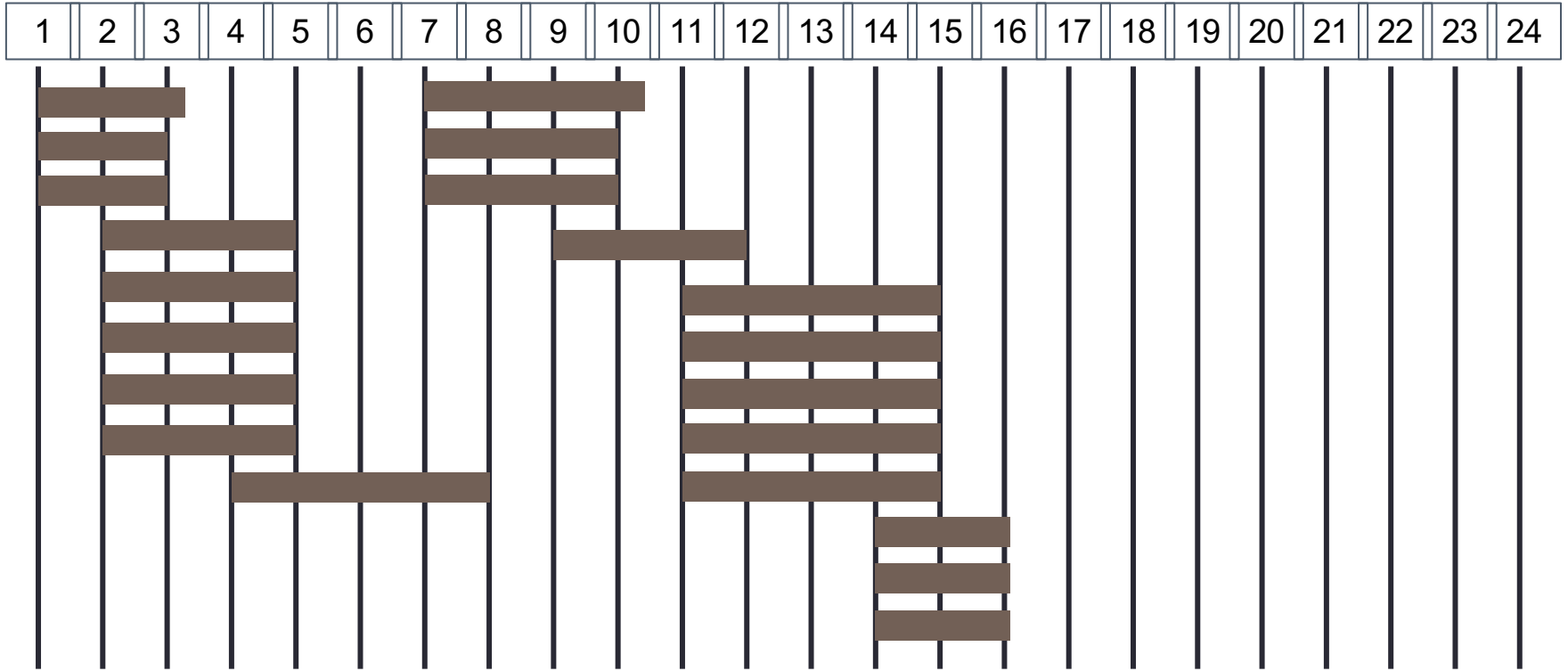
shortest duration



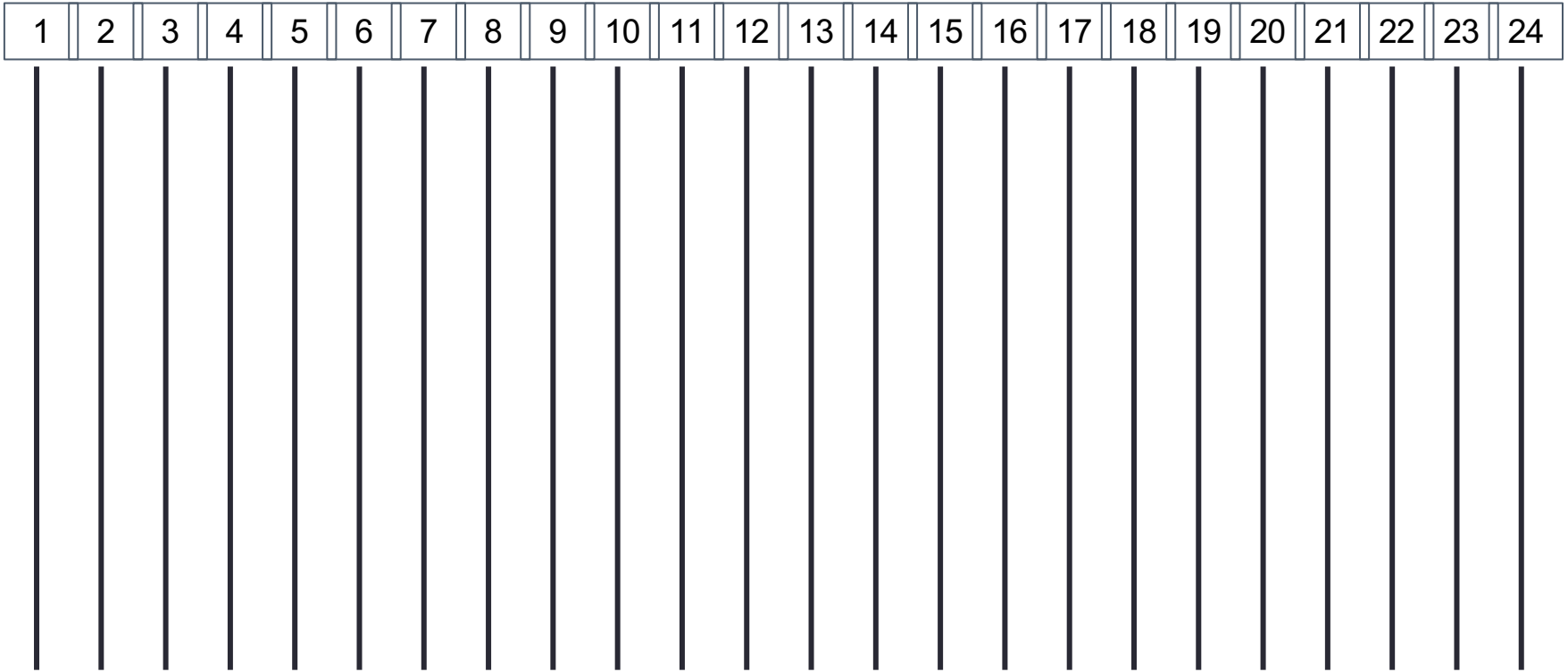
earliest start time



Counterexample for fewest conflicts



Earliest finish time

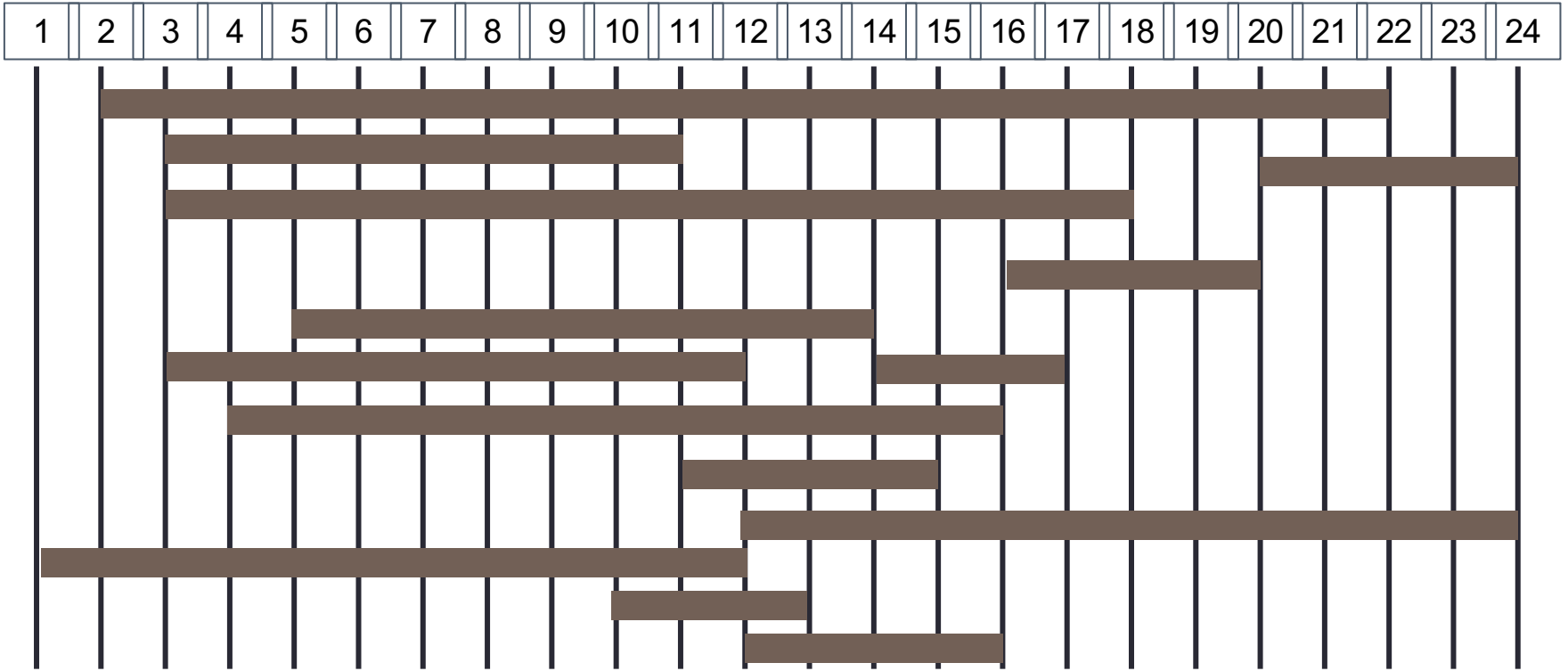


Event scheduling

- Your goal is to schedule the most events possible that day such that no two events overlap.

- Exponential is too slow. Let's try some greedy strategies:
  - ~~Shortest duration~~
  - ~~Earliest start time~~
  - ~~Fewest conflicts~~
  - Earliest end time (We can't find a counterexample!!)
- We will prove it works soon!!!!!

Event scheduling



PROVING OPTIMALITY

- We need to show:

For every instance I, let GS be the greedy algorithm's solution to I. Let OS be any other solution for I. Then

$$Value(OS) \leq Value\ (GS) \text{ or } Cost(GS) \leq Cost\ (OS)$$

- Tricky part: OS is arbitrary solution, not one that makes sense. We don't know much about it

TECHNIQUES TO PROVE OPTIMALITY

- We'll see a number of general methods to prove optimality:

1. Modify the solution , aka Exchange : most general
2. Greedy-stays-ahead: more intuitive
3. Greedy achieves the bound: also comes up in approximation, LP, network flow
4. (Unique or canonical local optimum). We might or might not have time to discuss this one.

Which one to use is up to you, but Modify-the-solution AKA Exchange argument applies almost universally, others can be easier but only work in special cases.

Opportunity costs generalized

- GS = First greedy move + greedy solution for rest
- OS= Other first move + some solution for rest

- Direct benefit = Value(First greedy move)- Value(other move)
- Opportunity cost: Optimal value for rest of OS – Optimal for rest of GS (or some bound on this)

Modify-the-solution AKA exchange argument

- General structure of modify-the-solution:
  - 1. Let g be the first greedy choice the algorithm makes.
  - 2. Let OS be a solution achieved by not choosing g.
  - 3. Show how to transform OS into some solution OS' that chooses g, and that is at least as good as OS.
  - 4. Use 1-3 in an inductive argument.

Modify-the-solution AKA exchange argument

- General structure of modify-the-solution:
  - 1. Let g be the first greedy choice the algorithm makes.
  - 2. Let OS be a solution achieved by not choosing g.
  - 3. Show how to transform OS into some solution OS' that chooses g, and that is at least as good as OS. (this is called the exchange argument)
    - Must show that
      - OS' is a valid solution and
      - OS' is better than OS.
  - 4. Use 1-3 in an inductive argument.
    - Base Case: the greedy strategy works for an instance of size 1.
    - Assume the greedy strategy works for all instances of size $< n$.
    - Let OS be any solution for $I, |I| = n$. Then by the EA there is another solution OS' such that $|OS| \leq |OS'|$ and OS' includes the first greedy choice g.

$$|OS(I)| \leq |OS'(I)| = |\{g\} \cup S(I')| \leq |\{g\} \cup GS(I')| = |GS(I)|$$

Modify-the-solution AKA exchange argument

- 1. Let g be the first greedy choice the algorithm makes.
- 2. Let OS be a solution achieved by not choosing g.
- 3. Show how to transform OS into some solution OS' that chooses g, and that is at least as good as OS.
  - A. Define OS'. We can use OS and g as ingredients.
  - B. Show that OS' is a valid solution, i.e., meets the constraints of the problem. We can use: definition of OS'; definition of g; that OS is a valid solution.
  - C. Compare objective for OS to that of OS'. We can use the definition of OS', definition of g.

Modify-the-solution AKA exchange argument

- 1. Let g be the first greedy choice the algorithm makes.
- 2. Let OS be a solution achieved by not choosing g.
- 3. Show how to transform OS into some solution OS' that chooses g, and that is at least as good as OS.
- 4. Use 1-3 in an argument by induction on input size.
  - Base Case: the greedy strategy works for an instance of size 1.
  - Assume the greedy strategy works for all instances of size $< n$.
  - Let OS be any solution for $I, |I| = n$. Then by step 3, there is another solution OS' such that $|OS| \leq |OS'|$ and OS' includes the first greedy choice g.

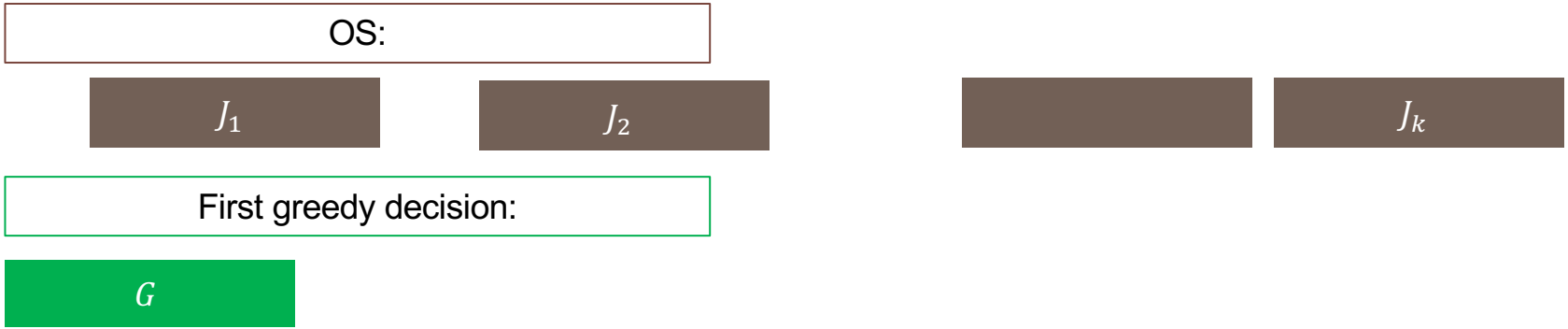
$$|OS(I)| \leq |OS'(I)| = |\{g\} \cup S(I')| \leq |\{g\} \cup GS(I')| = |GS(I)|$$

Earliest end time

- Correctness proof of earliest end time algorithm:
  - Let $E = \{E_1, \dots E_n\}$ be the set of all events with s_i, f_i the start and finish times of E_i.
- Exchange Argument:
  - Say G is the event with the earliest finish time. (First greedy decision: include G)
  - Let OS be a non-overlapping schedule that does not include G
  - **Claim:** there is a schedule OS' that does include G such that
    $$|OS'| \geq |OS|.$$
  - Proof: (exchange argument)
  - Let the events in OS be $J_1, \dots, J_k$, ordered by start and finish times ($J_1 \neq G$)
  - Define OS' from OS:

Exchange argument cont.

- What we have to start:

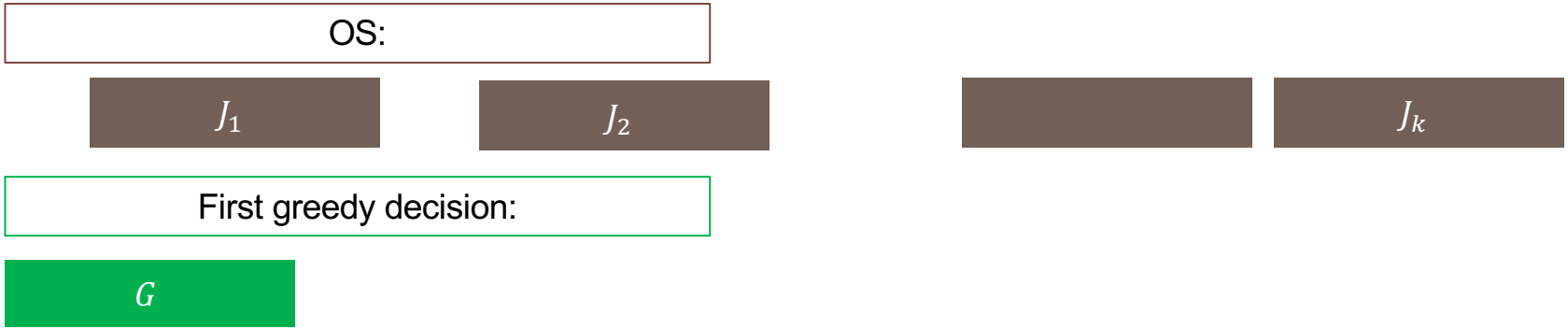


Our to-do list:

Define OS'

- OS' must have no overlaps
- OS' must include G
- $|OS'| \geq |OS|$

Define OS'



OS' = (OS – $\{J_1\}$) ∪ $\{G\}$

OS' has no overlaps

- OS' = (OS – $\{J_1\}$) ∪ $\{G\}$





- We must show two things:
  - OS' is valid (i.e., OS' does not have any overlapping events)
  - $|OS'| \geq |OS|$

OS' has no overlaps

- OS' = (OS – $\{J_1\}$) ∪ $\{G_1\}$



- **Claim:** OS' is valid (i.e., OS' does not have any overlapping events)

Proof: Since OS is valid. Any pair of J_i, J_ℓ events will not overlap. So it is sufficient to prove that G does not overlap with any of the J_i events. In particular: J_2.

Claim: G does not overlap with J_2. (prove this with your neighbors)

OS' has no overlaps

- $OS' = (OS - \{J_1\}) \cup \{G\}$



OS' is a valid solution!!!

It is clear why $|OS'| \geq |OS|$.

In fact: $|OS'| = |OS|$ which is fine.

INDUCTION

- What have we established?
- Exchange argument Claim:
  - Let g be the first greedy choice and let OS be any solution that does not include g. Then there exists some solution OS' that does include g and is "better" or "just as good" as OS.

- This alone does not prove that the algorithm is correct. It shows that the first move isn't incorrect. We will use it in an induction argument, to show later moves aren't incorrect.

INDUCTION

- The greedy solution is optimal.
- AKA: for any solution OS, the greedy solution GS is “better” or “just as good” as OS

INDUCTION

- The greedy solution is optimal.
- AKA: for any solution OS, the greedy solution GS is "better" or "just as good" as OS
  - Proof by strong induction on n, the number of events.
  - Base Case. $n = 1$. The greedy (actually, any) choice works.
  - Inductive Hypothesis (Strong.)
  - Suppose that $n > 1$ and assume that the greedy algorithm is optimal for any k events for $1 \leq k \leq n - 1$.
    - i.e. If $|I| < n$ then for any solution $OS(I)$, $|OS(I)| \leq |GS(I)|$.
  - Goal: Greedy is optimal for any n events

INDUCTION

Inductive Step: Let OS be any solution of the set of events $I = \{E_1, \ldots, E_n\}$.

(What can we conclude about OS using the exchange argument?)

INDUCTION

Inductive Step: Let OS be any solution of the set of events $I = \{E_1, \ldots, E_n\}$.

Then there exists a solution OS' such that $|OS| \leq |OS'|$ and OS' includes the first greedy choice G.

Let I' be the set of events that don't conflict with G.

Then $OS' = \{G\} \cup S(I')$ where $S(I')$ is some solution of I'.

- (What can we conclude using the inductive hypothesis?)

INDUCTION

Inductive Step: Let OS be any solution of the set of events $I = \{E_1, \dots, E_n\}$.

Then there exists a solution OS' such that $|OS| \leq |OS'|$ and OS' includes the first greedy choice G.

Let I' be the set of events that don't conflict with G.

Then $OS' = \{G\} \cup S(I')$ where $S(I')$ is some solution of I'.

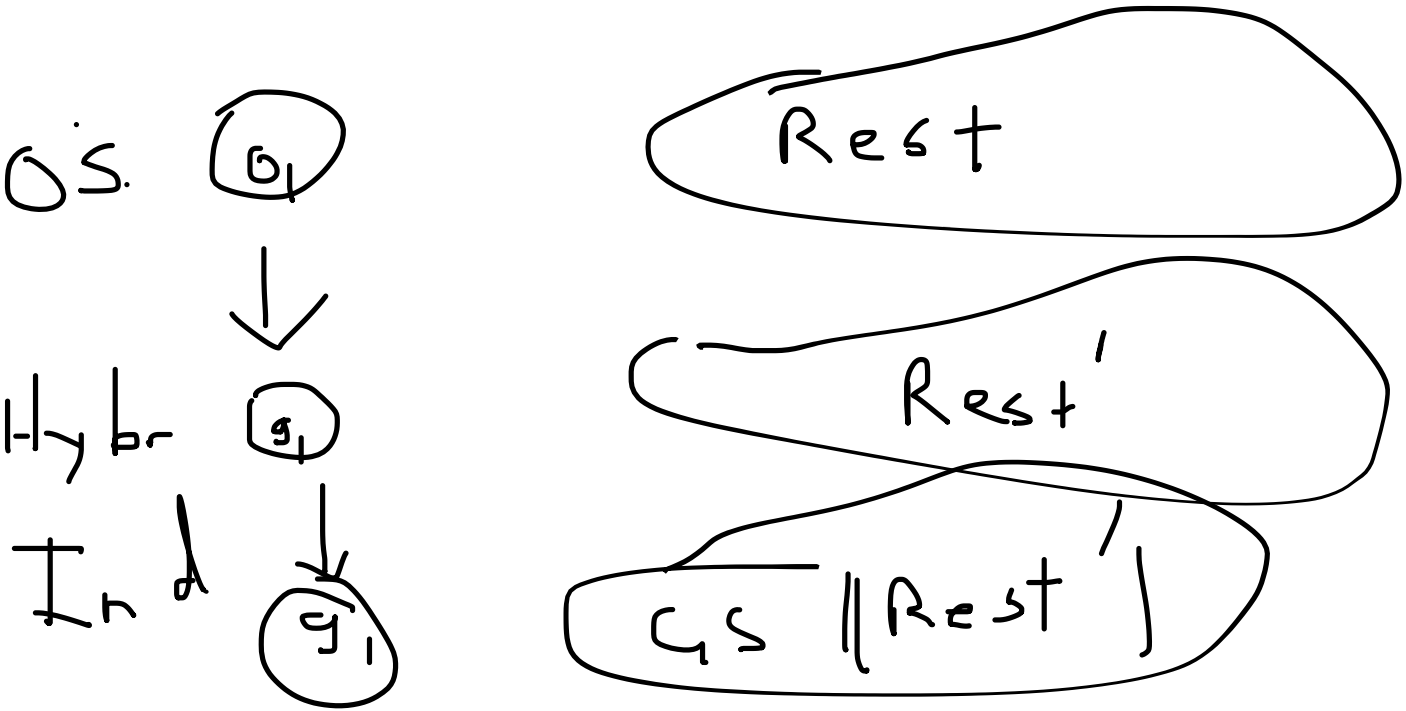
Since $|I'| < n$, by the inductive hypothesis, $|S(I')| \leq |GS(I')|$.

And by definition, $GS(I) = \{G\} \cup GS(I')$.

Conclusion:

$$|OS| \leq |OS'| = |\{G\} \cup S(I')| \leq |\{G\} \cup GS(I')| = |GS(I)|$$

Conclusion: The GS is optimal for every set of events


OS
Hybr
Ind
Rest
Rest'
GS || Rest'

Event Scheduling implementation

- Design an algorithm that uses the greedy choice of picking the next available event with the earliest finish time.
- Idea 1: Sort the events by non-decreasing finish time.
- Idea 2: Lazy delete: Don’t run through deleting conflicting events. Instead, keep a rule that says whether events conflict.

Rule

- Say first event to finish is (s_i, f_i). When do other events conflict?

Rule

- Say first event to finish is (s_i, f_i). When do other events conflict?

- Events that start before f_i conflict, because this event must be first, having first finish time.
- First event to not conflict is first I with $f_i \leq s_I$

Event Scheduling IMPLEMENTATION

- Initialize a Queue S.
- Sort the intervals by finish time.
- Put the first event E_1 in S.
- Set $F = f_1$
- For $i = 2 \ldots n$:
  - If $s_i \geq F$:
    - enqueue(E_i, S)
    - $F = f_i$
- Return S

Event Scheduling IMPLEMENTATION

- Initialize a Queue S.
- Sort the intervals by finish time. O(n log n)
- Put the first event E_1 in S.
- Set $F = f_1$
- For $i = 2 \ldots n$: O(n)
  - If $s_i \geq F$:
    - enqueue(E_i, S). O(1)
    - $F = f_i$
- Return S.
- Total is O(n log n)

Proving ftf is optimal via GREEDY STAYS AHEAD

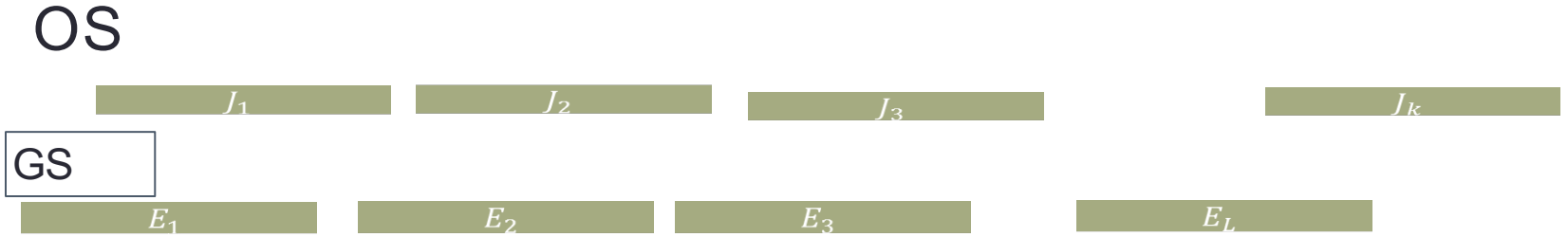
Compare all of GS to all of OS, instead of just first greedy move

OS



Show GS is at least as good as OS, in some suitable sense, *every step of the way.* *At least as good* needs interpretation. What should it mean here?

GREEDY STAYS AHEAD



Claim: Finish(E_i) ≤ Finish(J_i)

Proof by induction on i. True for E_1, because it is the first to finish.

E_{i+1}: This is the interval starting after Finish(E_i) with the earliest end time.

J_{i+1} also begins after Finish(E_i), since Finish(J_i) ≥ Finish(E_i).

Therefore Finish(J_{i+1}) ≥ Finish(E_{i+1}).

GREEDY STAYS AHEAD CONCLUSION

Contradiction: Assume |GS|=L < |OS|=k.

We know $Finish(E_L) \leq Finish(J_L) \leq Start(J_{L+1})$

But then we could append J_{L+1} to GS, i.e., GS wouldn't end with L events.

GREEDY STAYS AHEAD tEMPLATE

- Define progress measure (Not always clear how.)
- Order the decisions in OS to line up with GS – (Not always possible or easy)
- Prove by induction that the progress after the i'th decision in GS is at least as big as after ith decision in OS
- Conclude that GS is at least as good as OS.

ANOTHER ARGUMENT: FTF Achieves the bound

Consider the finish times $f_1,\ldots,f_k$ of the events FTF schedules.

Claim: every event contains one of these times.

- True for events in FTF.
- Let E = (s,f) be any event that FTF does not schedule. Let E_i be the first in FTF with $s \leq f_i$. (Must exist, or we'd put E_i at end in FTF).
- Since $s > f_{i-1}$, E wasn't deleted before E_i was added. So $f \geq f_i$, since E_i was the first to finish.
- Therefore, f_i is in E.

Since no two events in any solution can intersect, at most k events can be in any solution. Therefore, FTF is at least as large as any solution.

Event Scheduling with multiple rooms

Suppose you have a conference to plan with n events and you have an unlimited supply of rooms. How can you assign events to rooms in such a way as to minimize the number of rooms?

Brute Force:

- Certainly you won't need more than n rooms.
- So how many ways can you assign n events to n rooms?

Event Scheduling with multiple rooms

Suppose you have a conference to plan with n events and you have an unlimited supply of rooms. How can you assign events to rooms in such a way as to minimize the number of rooms?

Ideas for a greedy algorithm?

Event Scheduling with multiple rooms

Suppose you have a conference to plan with n events and you have an unlimited supply of rooms. How can you assign events to rooms in such a way as to minimize the number of rooms?

Greedy choice:

- Number each room from 1 to n.
- Sort the events by earliest start time.
- Put the first event in room 1.
- For events 2…n, put each event in the smallest numbered room that is available.

TECHNIQUES TO PROVE OPTIMALITY

Some general methods to prove optimality:

- Modify-the-solution, aka Exchange: most general
- Greedy-stays-ahead: often the most intuitive
- Greedy-achieves-the-bound: also used in approximation, LP, network flow
- Unique-local-optimum: dangerously close to a common fallacy

Which one to use is up to you.

Achieves-the-bound

1. Logically determine a bound on the value of the solution that must be satisfied by any valid answer.

2. Then show that the greedy strategy achieves this bound and therefore is optimal.

Achieves-the-bound

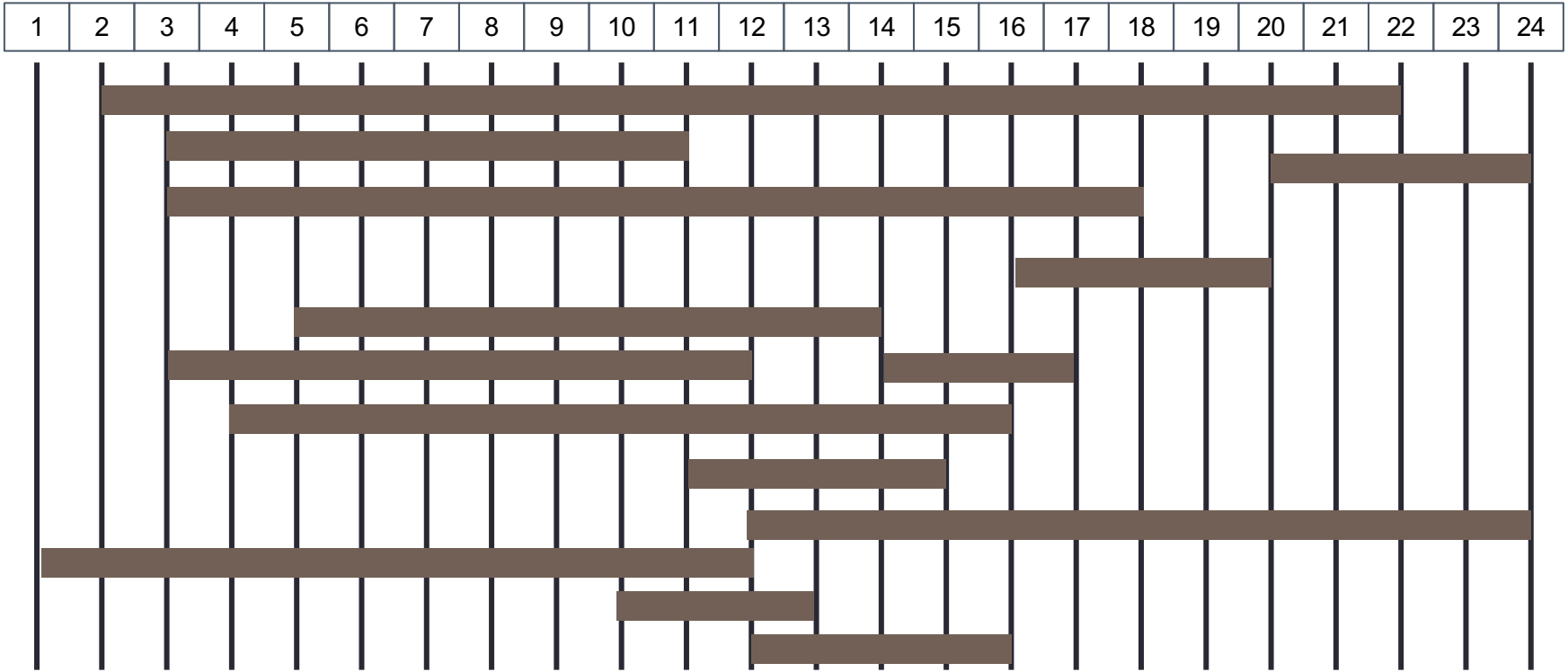
- Let t be any time during the conference.
- Let $B(t)$ be the set of events taking place at time t.

Bounding Lemma: Any valid schedule requires at least $|B(t)|$ rooms.

Proof:
There are $|B(t)|$ events taking place at time t.
They all need to be in different rooms.
So we need at least $|B(t)|$ rooms.

- Let $L = max(|B(t)|)$ over all t.
- Then L is a lower bound on the number of rooms needed.

Event scheduling



Achieves-the-bound

Achieves-the-Bound Lemma: Let k be the number of rooms picked by the greedy algorithm. Then at some point t, $|B(t)| \geq k$. In other words there are at least k events happening at time t.

Proof:

Let t be the starting time of the first event to be scheduled in room k.

Then by the greedy choice, room k was the least number room available at that time.

This means at time t, there was an event happening in rooms room 1, room 2, …, room $k-1$. And plus an event happening in room k

Therefore $|B(t)| \geq k$.

CONCLUSION: GREEDY IS OPTIMAL

- Let GS be the greedy solution.
- Let OS be any other schedule.

- Let L = max |B(t)| over all t.

- By the Bounding lemma, Cost(OS) ≥ L.
- By the achieves-the-bound lemma, Cost(GS) = |B(t)| ≤ L for some t.
- Putting the two together, Cost(GS) ≤ Cost(OS).

Achieves-the-bound

The way it works:

- Argue that when the greedy solution **reaches its peak cost**, it reveals a **bound**.
- Then show this bound is also a lower bound on the cost of any other solution.
- So we are showing: Cost(GS) ≤ Bound ≤ Cost (OS)

This is a proof technique that does not work in all cases.

Still to come:

- Other proof templates for greedy algorithms
- Minimum spanning tree
- Data structures for disjoint sets
- Amortized analysis for data structures
- Approximation algorithms and their analyses